

React Introduction

React, also known as **ReactJS**, is a popular and powerful JavaScript library used for building dynamic and interactive user interfaces, primarily for **single-page applications (SPAs)**. It was developed and maintained by Facebook and has gained significant popularity due to its efficient **rendering techniques**, **reusable components**, and active community support.

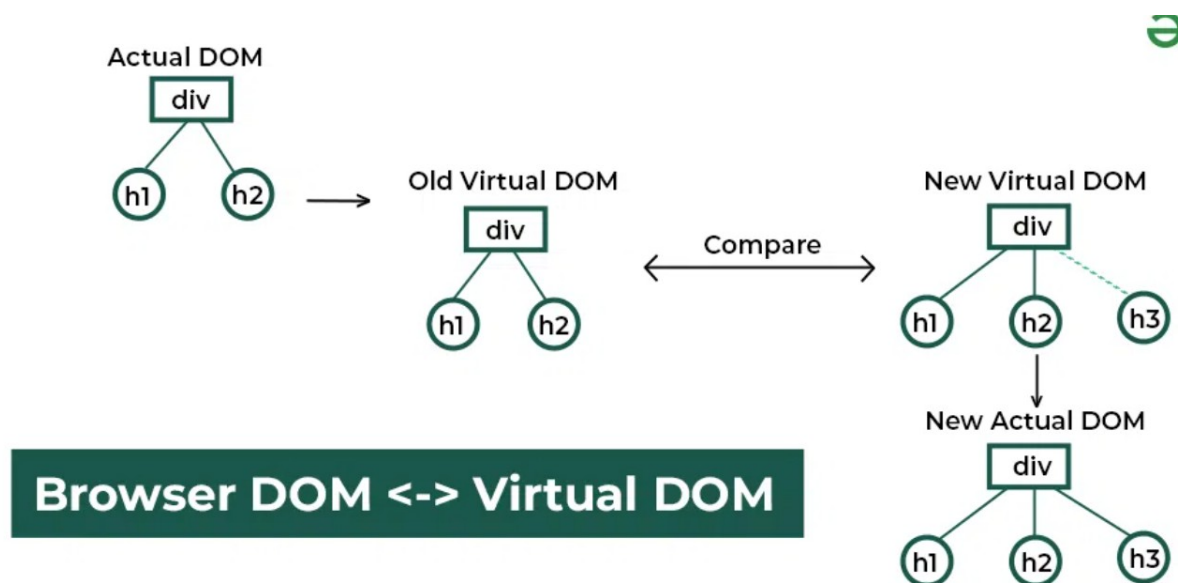
In this article, we will explore **React Introduction**, **what React is**, its key features, benefits, and why it's a great choice for modern web development.

History of React

- React was invented by Facebook developers who found the traditional DOM slow. By implementing a virtual DOM, React addressed this issue and gained popularity rapidly.
- The current stable version of ReactJS is 18.2.0, released on June 14, 2022. The library continues to evolve, introducing new features with each update

How does React work?

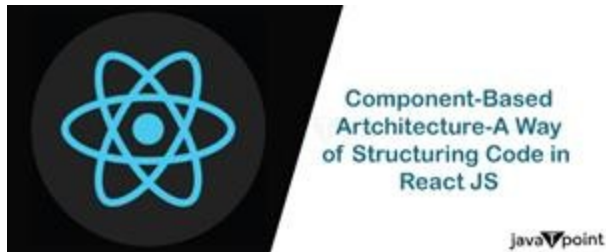
- React operates by creating an in-memory virtual DOM rather than directly manipulating the browser's DOM. It performs necessary manipulations within this virtual representation before applying changes to the actual browser DOM. React is efficient, altering only what requires modification.



Features of React

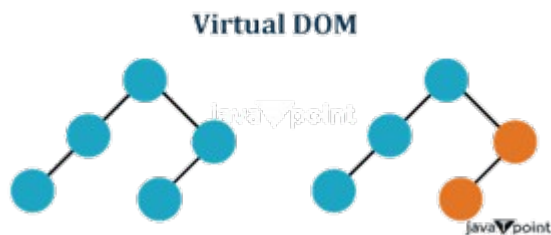
Here are some key features of React:

1. Component-Based Architecture:



React is centered around reusable components. A component is a self-contained, independent building block of a UI. These components can be composed together to create complex UIs. This component-based approach promotes reusability, maintainability, and a modular code structure.

2. Virtual DOM:



React uses a Virtual DOM (Document Object Model) to optimize UI updates. Instead of directly manipulating the actual DOM, React builds a virtual representation of it in memory. When changes occur in the application's state, React updates the virtual DOM first and then efficiently updates the real DOM to reflect the changes. This approach minimizes unnecessary DOM manipulations and enhances performance.

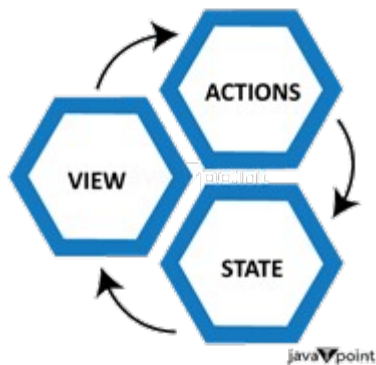
3. Declarative Syntax:

React promotes a declarative style of programming, where developers describe how the UI should look based on the current state of the application. React then automatically updates the UI to match this description, reducing the need for manual DOM manipulation and making it easier to reason about the UI's behavior.

4. JSX (JavaScript XML):

React uses JSX, a syntax extension for JavaScript that allows developers to write HTML-like code within their JavaScript files. JSX is transpiled into regular JavaScript for browser compatibility.

5. Unidirectional Data Flow:



React enforces a unidirectional data flow. It means that data flows in a single direction through the application. This helps to maintain a predictable and traceable flow of data, making it easier to understand how changes in data affect the UI.

6. Reusable Components:



React encourages the creation of reusable UI components. Developers can build UI components once and reuse them across different application parts, leading to more efficient development and easier maintenance.

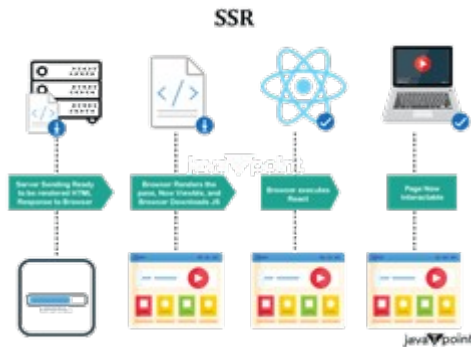
7. React Ecosystem:



React has a rich ecosystem of libraries and tools that complement its capabilities. These include libraries for routing (React Router), state

management (Redux, Mobx), server-side rendering (Next.js), and more. These tools extend React's functionality and enable developers to build full-featured applications.

8. Server-Side Rendering (SSR):



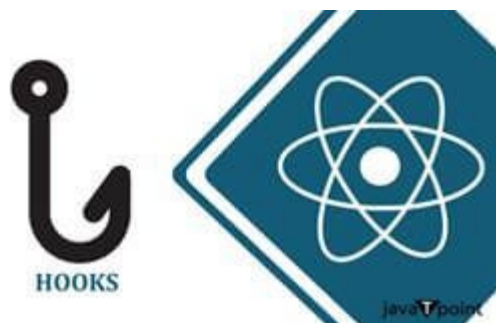
React can be used for server-side rendering (SSR), a technique where the initial HTML for a web page is generated on the server rather than in the browser. SSR can improve performance and SEO by providing search engines with pre-rendered content.

9. Mobile Development:



React Native is a framework built on top of React that allows developers to create native mobile applications for iOS and Android using JavaScript and React components. This enables code sharing between web and mobile apps, streamlining development efforts.

10. Hooks:



Introduced in React 16.8, hooks allow functional components to manage state, side effects, and other features previously reserved for class

components. Hooks, like `useState`, `useEffect`, and `useContext`, make it easier to work with state and lifecycle in functional components

React Lifecycle

The React lifecycle is an essential concept for any React developer to understand. Each component in React has a lifecycle that you can monitor and manipulate during its three main phases: Mounting, Updating, and Unmounting. These phases represent the stages a React component goes through from its creation to its removal from the DOM.

React components follow a predictable lifecycle, making it easier for developers to manage data fetching, subscriptions, DOM updates, and other side effects at appropriate times. In this article, we will explore the three phases of the React lifecycle and demonstrate how to use lifecycle methods effectively in class-based components.

Table of Content

- [React Lifecycle Phases](#)
- [Phases of Lifecycle in React Components](#)
- [Implementing the Component Lifecycle methods](#)

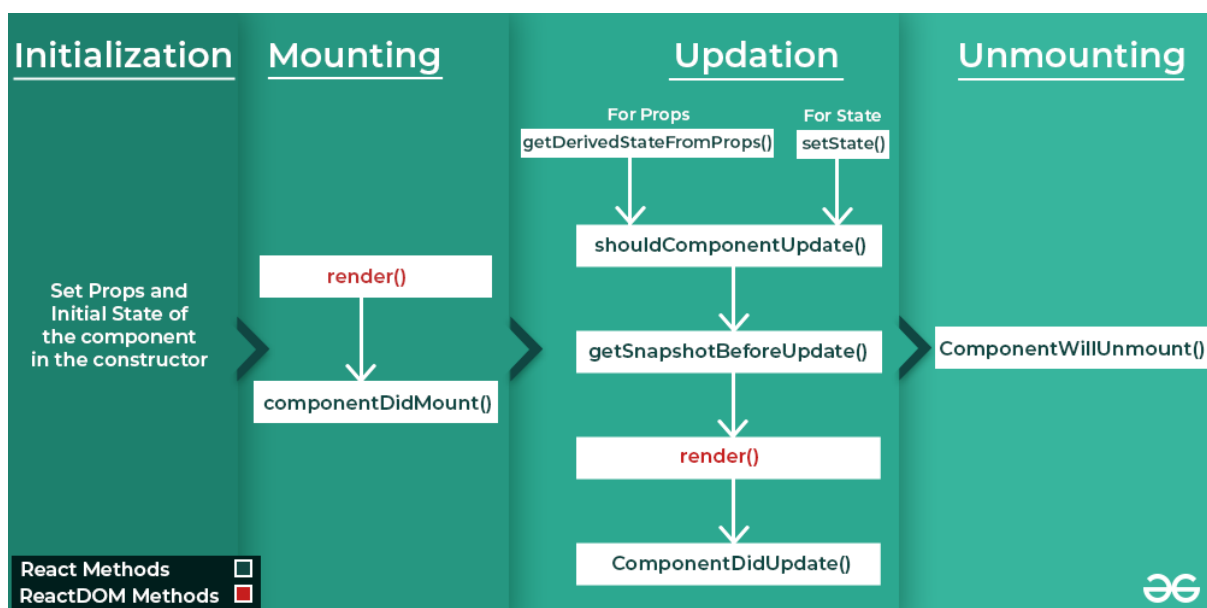
React Lifecycle Phases

React Lifecycle is defined as the series of methods that are invoked in different stages of the component's existence. The definition is pretty simple but what do we mean by different stages? A [React](#) Component can go through four stages of its life as follows.

The React lifecycle is divided into three main phases:

1. Initialization phase: This is the stage where the component is constructed with the given Props and default state. This is done in the constructor of a Component Class.
2. Mounting Phase: This phase begins when a component is created and inserted into the DOM.
3. Updating Phase: This occurs when a component is re-rendered due to changes in props or state.
4. Unmounting Phase: This is the final phase when a component is removed from the DOM.

React provides the developers with a set of predefined functions that if present are invoked around specific events in the lifetime of the component. Developers are supposed to override the functions with the desired logic to execute accordingly. We have illustrated the gist in the following diagram.



Now let us describe each phase and its corresponding functions.

Phases of Lifecycle in React Components

To gain a deeper understanding of these concepts and enhance your React skills, the [React JS Course](#) offered by GeeksforGeeks provides a thorough foundation. This course covers not only the lifecycle methods but also the latest advancements in React, including hooks and their lifecycle effects. By enrolling in this course, you'll be equipped with the practical skills and knowledge needed to create efficient and effective React applications.

Install ReactJS on Windows

you have the following essentials in place:

Administrator Access

Administrator privileges are required to install software and make system-wide changes on your Windows machine.

- Installing tools like Node.js and npm modifies system files and environment variables.
- Without administrator access, you might face permission errors during the installation process.

To check if you have administrator access: Right-click on the Start Menu and select Settings > Accounts > Your Info

Steps to Install React on Windows

Follow these Steps to Install React.js and [Create a React App](#) on Windows 10/11:

React relies on [Node.js](#) and its [package manager](#), npm, to manage dependencies

. To install React, start by installing **Node.js**, which includes npm (Node Package Manager).

- Visit the [official Node.js website](https://nodejs.org/).
- Download the **LTS (Long Term Support)** version for Windows. This version is stable and recommended for most users.

Step 2: Run the Installer

- Locate the downloaded .msi file and double-click to run it.
- Follow the prompts in the setup wizard, accept the license agreement, and use the default settings for installation.

Step 3: Verify Node.js Installation

- Open **Command Prompt** or **PowerShell** > Check the installed versions by running these commands:
- Type **node -v** and press Enter to check the Node.js version.
- Type **npm -v** and press Enter to check the npm version.
- Both commands should return version numbers, confirming successful installation.

Step 4: Install CRA (Deprecated Alternative)

- **Create React App** (CRA) has been officially deprecated in 2023 and is no longer the recommended method for starting React projects. Instead, tools like **Vite** or **Next.js** are now preferred for creating React applications due to better performance and modern features. However, if you still want to use it, follow these steps:

- **1. Install CRA Globally**
- `npm install -g create-react-app`

```
PS C:\Users\GFG0512> npm install -g create-react-app
npm warn deprecated fstream-ignore@1.0.5: This package is no longer supported.
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to
coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated uid-number@0.0.6: This package is no longer supported.
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm warn deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported
npm warn deprecated fstream@1.0.12: This package is no longer supported.
npm warn deprecated tar@2.2.2: This version of tar is no longer supported, and will not receive security updates. Please upgrade asap.

changed 64 packages in 1s

4 packages are looking for funding
  run 'npm fund' for details
PS C:\Users\GFG0512>
```


2. Verify the installation

It will globally install react app for you. To check everything went well run the command

create-react-app -version

```
b2 C:/Users/GFG0512/Desktop>
2'0'T
b2 C:/Users/GFG0512/Desktop> create-react-app --version
```

Step 5: Create Directory for React Projects

1. Create a New Folder

Create a new folder where you want your React app, use the following command:

mkdir newfolder

```
PS C:\Users\GFG0512\Desktop> mkdir newfolder

Directory: C:\Users\GFG0512\Desktop

Mode                LastWriteTime         Length Name
----                -
d-----          10-09-2024    11:00                newfolder

PS C:\Users\GFG0512\Desktop>
```

2. Move inside the folder

Move inside the same folder using the below command:

cd newfolder (your folder name)

```
PS C:\Users\GFG0512\desktop> cd newfolder
PS C:\Users\GFG0512\desktop\newfolder>
```

Step 6: Create the React App Using CRA Command

If you're using CRA:

Run the following command to create a React app:

`npx create-react-app reactfirst`

```
PS C:\Users\GFG0512\desktop\newfolder> npx create-react-app reactfirst

Creating a new React app in C:\Users\GFG0512\Desktop\newfolder\reactfirst.

Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts with cra-template...

added 1486 packages in 3m

263 packages are looking for funding
  run `npm fund` for details

Initialized a git repository.

Installing template dependencies using npm...

added 63 packages, and changed 1 package in 8s

263 packages are looking for funding
  run `npm fund` for details
Removing template package using npm...
```

It will take some time to install the required dependencies

Note: Due to npm naming restrictions, names can no longer contain capital letters, thus type your App name in lowercase.

```
C:\Users\sa552>create-react-app FirstReact
Cannot create a project named "FirstReact" because of npm naming restrictions:

  * name can no longer contain capital letters

Please choose a different project name.
```

If using Vite:

Use the following command

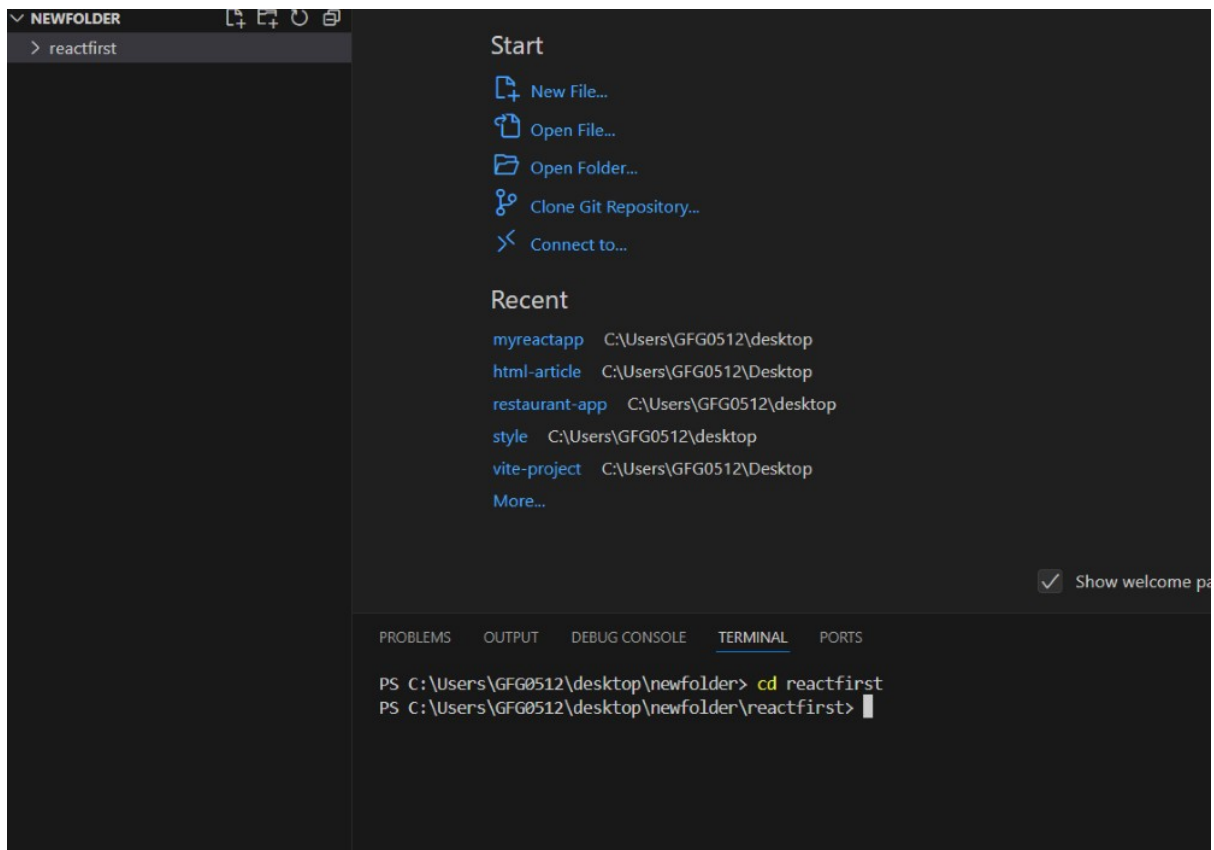
```
npm create vite@latest reactfirst -- --template react
```

Step 7: Open the Project in a Code Editor

Now open the [IDE](#) of your choice for eg. Visual studio code and open the folder where you have installed the react app **newfolder** (in the above example) inside the folder you will see your app's name **reactapp** (In our example). Use the terminal and move inside your app name folder.

Use the following command:

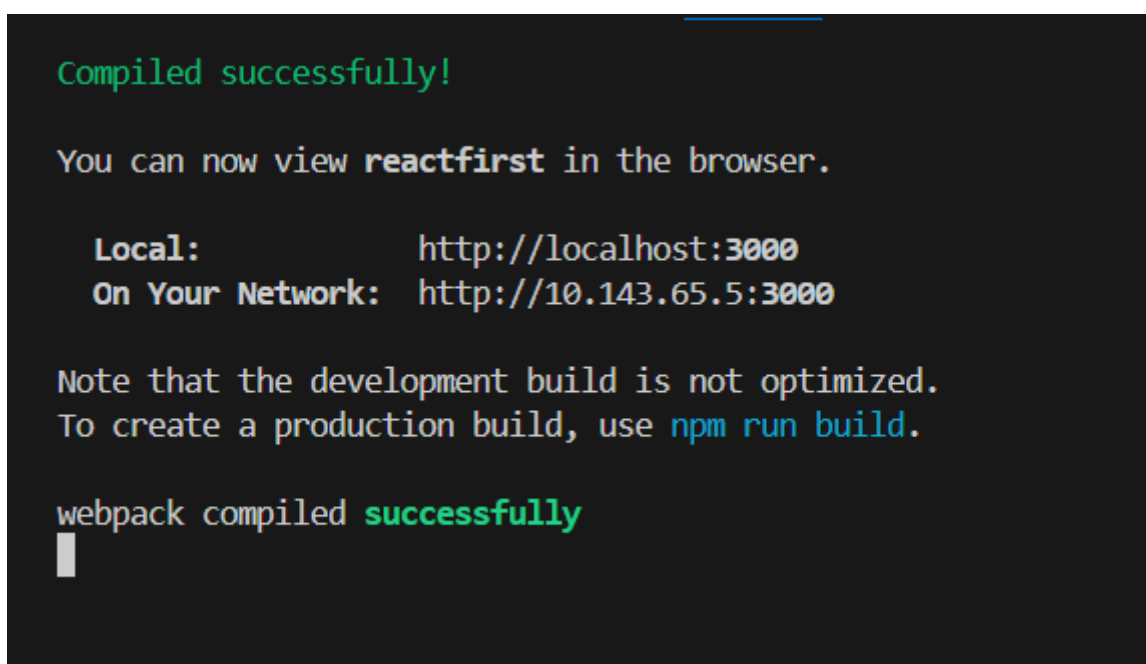
```
cd reactapp (your app name)
```



Step 8: Run the React App

Start the development server by running:

`npm start`



ReactJS Babel Introduction

Babel is an open-source JavaScript compiler and transpiler that allows using the latest JavaScript(ES6+) features, ensuring compatibility across older and newer browsers. Babel enables React developers to write clean, modern syntax without worrying about browser limitations.

How Babel Works with ReactJS

[React](#) applications often use [JSX](#) syntax to define components. Since browsers don't natively understand JSX, Babel converts it into standard JavaScript functions using the `@babel/preset-react` plugin. This step is very important for seamless React development.

- Parsing: Babel reads the source code and converts it into an Abstract Syntax Tree (AST).
- Transformation: Applies transformations to the AST using presets and plugins to handle JSX and ES6+ features.
- Code Generation: Converts the transformed AST back into JavaScript code.

Steps to Use Babel with React

Step 1: Create a directory for the project.

```
mkdir my-app
```

```
cd my-app
```

Step 2: Initialize the application using the following command.

```
npm init -y
```

Step 3: Install the required React dependencies

```
npm i react react-dom
```

Step 4: Install webpack and babel using the command

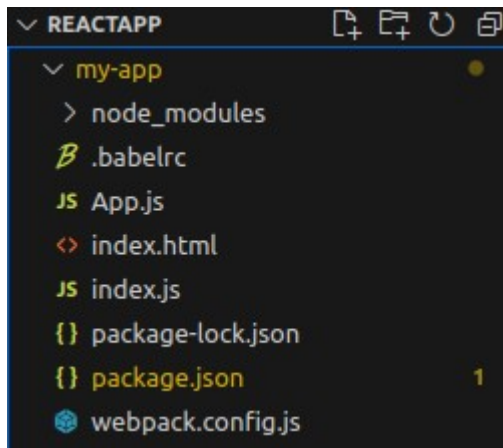
```
npm i webpack webpack-cli @babel/core  
@babel/preset-env @babel/preset-react babel-  
loader html-webpack-plugin webpack-dev-server  
-save-dev
```

Step 5: Inside the scripts section of package.json file add the following code

```
"scripts": {  
  "start": "webpack-dev-server --mode  
development --open",  
  "build": "webpack --mode production"  
}
```

Step 6: Create the files named index.html, App.js, index.js, webpack.config.js, .babelrc

Folder Structure



Dependencies

```
{  
  "scripts": {  
    "start": "webpack-dev-server --mode  
development --open",  
    "build": "webpack --mode production"  
  },  
  "dependencies": {  
    "babel-core": "^6.26.3",  
    "babel-preset-env": "^1.7.0",  
    "babel-preset-react": "^6.24.1"  
  },  
  "devDependencies": {  
    "@babel/core": "^7.22.9",  
    "@babel/preset-env": "^7.22.9",  
    "@babel/preset-react": "^7.22.5",  
  }  
}
```



```
"babel-loader": "^9.1.3",
"html-webpack-plugin": "^5.5.3",
"react": "^18.2.0",
"react-dom": "^18.2.0",
"webpack": "^5.88.2",
"webpack-cli": "^5.1.4",
"webpack-dev-server": "^4.15.1"
}
}
```

Step 7: Add the following code in index.html, index.js, and App.js

HTMLJavaScriptJavaScriptJavaScript

```
<!--index.html-->
<html>
<head>
</head>
<body>
  <div id="root"></div>
</body>
</html>
```

Step 8: Inside the .babelrc file add the following code

```
{  
  "presets": ["@babel/preset-env",  
"@babel/preset-react"]  
}
```

Step 9: To run the application type the following command in a web browser

npm start

Output

Hello Geeks

Key Features of Babel

- **ES6+ Compatibility:** Converts ES6+ syntax (e.g., arrow functions, classes) into ES5 syntax for older browsers.
- **JSX Compilation:** Transforms JSX ([JavaScript XML](#)) syntax used in React into regular JavaScript functions.
- **Plugin Support:** Extensible with plugins to add custom transformations and support experimental features.
- **Polyfilling:** Introduces support for missing APIs in older browsers via libraries like [@babel/polyfill](#).

- Code Optimization: Removes unused code and optimizes the final bundle for production environments.

Using Vite build tool

Steps to Set Vite by Manually Setting Up Vite in an Existing Project

1. Step 1: Install Vite as a dependency. `npm install vite --save-dev.`
2. Step 2: Create a `vite.config.js` file to configure Vite.
`import { defineConfig } from 'vite'; ...`
3. Step 3: Update our `package.json` scripts. `{ ...`
4. Step 4: Run the development server. `npm run dev.`